

Grounded Agentic AI based Troubleshooting Assistant

Team IISc

Indian Institute of Science (IISc)

Abstract

We present a multi-tenant, agentic retrieval-augmented generation platform for customer onboarding and support. The system lets each business upload proprietary documents once and serve grounded question answering over Web, WhatsApp, Slack, Teams, and MCP clients. A thin FastAPI gateway owns authentication, file validation, storage, webhooks, and tenant boundaries, while n8n workflows implement parsing, OCR, chunking, embedding, vector search, reranking, answer generation, and faithfulness checks. PostgreSQL with pgvector stores both persistent tenant knowledge and session-scoped ephemeral chunks. The final system includes admin and chat portals, channel adapters, self-service onboarding, an evaluation harness spanning 148 cases in 13 domains, and automated tests for backend, frontend, workflow, and channel behavior.

Grounded Agentic AI based Troubleshooting Assistant - System Architecture

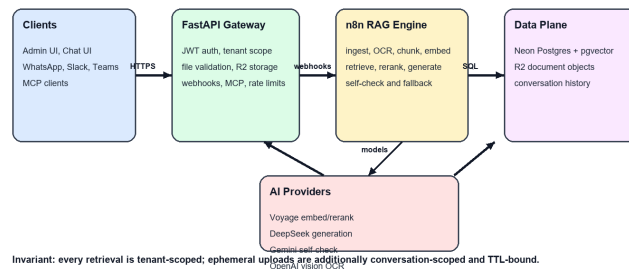


Figure 1. End-to-end architecture: clients enter through a thin FastAPI gateway; all RAG logic is delegated to n8n; tenant-scoped vectors live in pgvector.

1. Project Goal and Scope

Customer onboarding teams repeatedly answer questions already present in manuals, SOPs, policy files, and FAQs. The project implements a multi-tenant SaaS platform where a business uploads its knowledge base once, and customers receive grounded answers with citations through the channel they already use. The work is not a generic chatbot: the core contract is that answers should be traceable to uploaded documents, scoped to the correct tenant, and refused when the answer is absent.

The delivered scope covers tenant registration, role-based JWT authentication, admin document management, persistent and ephemeral ingestion, web chat with citations and conversation history, WhatsApp, Slack, Teams, MCP access, n8n retrieval workflows, database migrations, deployment documentation, and a RAGAS-based evaluation suite.

2. Architecture

The design deliberately separates product control from AI orchestration. FastAPI handles users, tenants, files, routing, rate limits, and security-sensitive webhooks. It does not embed, rerank, or generate text. n8n owns the AI pipeline: file extraction, OCR, 512-token chunking with overlap, Voyage embeddings, pgvector search, Voyage reranking, DeepSeek answer generation, Gemini faithfulness checks, and fallback handling.

PostgreSQL with pgvector is the single metadata and vector store. Persistent tenant knowledge is stored in `document_chunks`; one-off user uploads are stored in `ephemeral_chunks` with `tenant_id`, `conversation_id`, and `expires_at`. Cloudflare R2 stores uploaded objects, while Neon/Railway/Vercel provide the intended hosted footprint.

Architectural Invariants

- FastAPI is a product and security gateway, not an LLM orchestration layer.
- n8n owns all AI-side work: extraction, OCR, chunking, embedding, retrieval, reranking, generation, checks, and fallback.
- All persistent knowledge retrieval is tenant-scoped; ephemeral retrieval is tenant-scoped, conversation-scoped, and expiry-scoped.
- The user-facing contract returns source evidence, follow-up questions, and faithfulness metadata, allowing UI display and offline evaluation.
- The same knowledge base is reused across Web, WhatsApp, Slack, Teams, and MCP rather than creating channel-specific silos.

Core Request Flows

Admin upload: validate file or URL, store object metadata, trigger n8n ingestion, write chunks and embeddings, then receive an ingestion-status callback.

Question answering: authenticate and resolve tenant, load conversation history, call the retrieval workflow, save user and assistant messages, and return cited answers.

Ephemeral upload: accept a temporary file from chat, index it into `ephemeral_chunks`, answer only within that conversation, and purge on timeout or session end.

3. Work Delivered Across the Stack

Layer	Concrete implementation
Frontend	Next.js admin portal, chat portal, auth context, API clients, status badges, citations panel, conversation sidebar, mobile-ready layouts.
Backend	FastAPI routers for auth, admin, chat, webhooks, onboarding, mock pipeline, and MCP; async SQLAlchemy sessions; service layer for storage, validation, n8n, messages, and bots.
RAG workflows	n8n ingestion, retrieval, ephemeral ingestion, ephemeral retrieval, and purge workflows, with credentialized model/provider calls.
Data	Alembic migrations, pgvector schema, HNSW indexes, tenant/user/document/conversation/message models, upload audit, and channel mapping tables.
Ops/docs	Docker compose, Railway/Vercel/Neon deployment notes, demo notes, module specs, OpenAPI contract, and workflow documentation.

This branch therefore documents an implemented system rather than only a concept. The application code, workflow JSON, tests, evaluation corpora, and generated reports all exist in the repository and were cross-read while preparing this report.

4. Features Implemented

Subsystem	Work completed
Auth and tenancy	JWT login, /me, protected register, roles, tenant scoping, login rate limits.
Admin portal	Document upload and URL ingestion, document status, detail page, users, tenants, settings, route guards.
Chat portal	Conversation list, message history, citations panel, follow-up chips, faithfulness badge, empty/error states, theme toggle.
Channels	Web, WhatsApp/Twilio, Slack Events API, Microsoft Teams Bot Framework, and MCP JSON-RPC tools.
Ingestion	Persistent admin path to R2 and document_chunks; ephemeral chat path to conversation-scoped TTL chunks.
Retrieval	Vector top-k, rerank, grounded answer synthesis, source return, clarifications/refusals, saved chat messages.
Evaluation	148-case suite across 13 applications, RAGAS scoring, batched public webhook runner, HTML/CSV/JSON/Markdown reports.

5. Two Upload and Retrieval Paths

Path	Storage and scope	Purpose
Admin persistent	Cloudflare R2 plus document_chunks; visible to all users in the tenant until deleted.	Permanent business knowledge base for manuals, SOPs, FAQs, URLs, and reference guides.
Ephemeral chat	ephemeral_chunks with tenant_id, conversation_id, and 1-hour TTL; purgeable on session end.	One-off user uploads such as invoices, screenshots, or temporary support files without polluting the tenant corpus.

Both paths converge on the same principle: retrieval must return source evidence. Persistent documents support organization-wide self-service, while ephemeral uploads support private, session-scoped Q&A. The two paths share model infrastructure but differ in persistence, visibility, quotas, and cleanup.

6. Channel and Interface Coverage

- Admin web UI: document upload, URL ingestion, status polling, document detail, users, tenants, settings, and guarded routes.
- Chat web UI: conversation history, assistant/user message layouts, citations, follow-up chips, faithfulness states, and retry/error surfaces.
- WhatsApp: Twilio webhook validation, phone-to-tenant mapping, text responses, duplicate handling, reset flow, and media upload indexing.
- Slack: Events API handling, signature verification, thread replies, duplicate event protection, user onboarding, and shared message service.
- Teams: Bot Framework activity handling, tenant mapping, mention stripping, token validation, reset flow, and attachment indexing.
- MCP: JSON-RPC tool discovery and calls for knowledge-base queries and document listing from local clients.

7. Retrieval and Data Safety

Every retrieval path filters by tenant_id. Ephemeral retrieval additionally filters by conversation_id and expires_at > NOW(), so uploaded session files cannot leak into another tenant, another conversation, or the permanent knowledge base. Uploads use MIME and magic-byte validation, file-size limits, per-tenant hourly quotas, Twilio signature verification, n8n callback tokens, and a purge workflow for session end.

The user-facing response schema returns answer, sources, follow_up_questions, faithfulness, requires_clarification, and metadata. This schema powers the chat UI and enables evaluation because the retrieved evidence is preserved with assistant messages.

8. Security Controls

- Tenant isolation is repeated at API dependencies, SQL WHERE clauses, workflow queries, and channel mapping tables.
- Uploads are capped at 25 MB for admin documents and 10 MB for chat uploads, with per-tenant storage and hourly upload limits.
- Webhook trust boundaries use Twilio signatures, Slack signing secrets, Teams token validation, n8n callback tokens, and MCP API keys.
- Ephemeral data has two deletion paths: immediate purge on session end and an hourly TTL cleanup function in Postgres.

9. Evaluation

Evaluation Summary - Full Local RAGAS Run

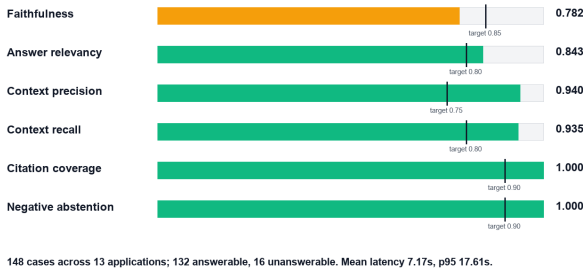


Figure 2. Local full-suite evaluation. Faithfulness remains below the configured threshold; retrieval and abstention indicators are strong.

The full local source-returning n8n run evaluated 148 cases across 13 applications: 132 answerable and 16 unanswerable. It achieved citation coverage 1.000, negative abstention 1.000, answer relevancy 0.843, context precision 0.940, and context recall 0.935. Faithfulness was 0.782, below the target 0.850, which is the main measured quality gap. Mean latency was 7.17 s and p95 latency was 17.61 s in the local run.

A backend DB-backed five-case run verified that /chat/query can call n8n, return answers, and save source objects into chat_messages.sources; it scored answer relevancy 0.940, context precision 0.918, and context recall 1.000, but again showed faithfulness below target. Public Railway smoke tests confirmed source-returning behavior and metadata coverage, with batching recommended for the full public RAGAS run.

10. Test Coverage

The repository contains 170 Python tests and 16 frontend component tests. Backend tests cover auth, tenant isolation, admin document APIs, upload validation, R2/local storage, conversation CRUD, Slack, WhatsApp, Teams, ingestion behavior, dataset validation, and evaluator reporting. Frontend tests cover chat panels, citations, conversation refresh, faithfulness badges, message layout, and theme behavior.

11. Main Findings

- The product architecture is modular: the gateway stays thin while n8n workflows can evolve independently.
- The strongest current behavior is retrieval: citation coverage, abstention, context precision, and context recall pass their targets in the full local run.
- The main quality improvement is answer faithfulness. The measured gap motivates stricter grounded prompts, lower-temperature synthesis, final validation, and fallback regeneration.
- The public endpoint works but should be evaluated in small deterministic batches because long full-suite runs are brittle on hosted infrastructure.

12. What the Report Claims

The report claims that the platform is feature-complete for the course-project scope and has credible local RAGAS evidence. It does not claim that the hosted public deployment has completed a full RAGAS pass, nor that faithfulness has reached the configured threshold. This distinction matters: the system retrieves and cites well, while final answer validation is the next engineering priority.

Evaluation Method

- Synthetic corpora cover 13 realistic application domains, including Kubernetes troubleshooting, support, compliance, onboarding, incident response, and legal navigation.
- Answerable cases include reference answers, expected source documents, and verbatim reference contexts; unanswerable cases measure abstention.
- The runner records answers, contexts, source metadata, latency, retries, and then computes RAGAS metrics where enabled.
- Generated artifacts include JSON, CSV, Markdown, and HTML so results can be audited outside the UI.

Quality Interpretation

Retrieval quality is strong: context precision, context recall, citation coverage, and abstention all pass their gates in the full local run. Answer faithfulness is the exposed risk: the system usually retrieves the right evidence, but some generations still add or over-compress claims.

Next Engineering Work

- Tighten answer prompts and require claim-by-claim support before returning final text.
- Run the public Railway evaluation in deterministic batches and consolidate the batch reports.
- Add the Gemini self-check and fallback path to the lean ephemeral retrieval workflow.
- Add hybrid keyword plus vector retrieval and structure-aware chunking with stronger page metadata.

Supplementary Material

The first three pages summarize the complete project. This supplement records implementation evidence, contracts, evaluation detail, and module-level work so the main paper stays readable while the report remains complete.

A. Repository Evidence Read

Area	Evidence files
Project/spec	README.md, PROJECT_SPEC.md, ARCHITECTURE.md, SKILLS.md, TEAM_WORKFLOW.md
Backend	backend/app/main.py, api/auth.py, api/admin.py, api/chat.py, api/webhooks.py, services/*, bots/*, mcp/*
Database	database/init.sql, backend/alembic/versions/*, backend/app/models/models.py
n8n	ingestion-pipeline.json, retrieval-pipeline.json, ingest/retrieve/purge ephemeral workflows
Frontend	frontend/src/app/admin/*, frontend/src/app/chat/*, frontend/src/components/chat/*, frontend/admin/*,md
Evaluation	evaluation/*.py, application_suite.jsonl, sample-data/*, artifacts/evaluation_results/*

B. API and Workflow Contracts

Admin uploads call POST /admin/documents/upload or POST /admin/documents/url. FastAPI validates and stores metadata, then triggers n8n /webhook/ingest with document_id, tenant_id, source_type, source_url, and title. n8n calls back to /webhooks/n8n/ingestion-status with status, chunk_count, error_message, and callback_token.

Chat uses POST /chat/query with query and optional conversation_id. The unified message service creates or loads the conversation, passes recent history to the pipeline, saves user and assistant messages, and returns answer, sources, follow-up questions, faithfulness, clarification state, and conversation metadata.

Ephemeral session RAG uses /webhook/ingest-ephemeral, /webhook/retrieve-ephemeral, and /webhook/purge-ephemeral. The read path searches only rows matching tenant_id, conversation_id, and expires_at > NOW(); the delete path includes tenant_id in the WHERE clause for cross-tenant purge defense.

C. Database and Security Model

- Core tables: tenants, users, documents, document_chunks, conversations, chat_messages, upload_audit, whatsapp_tenant_map, teams_tenant_map.
- Vector tables use pgvector vector(1024) and HNSW indexes with vector_cosine_ops.
- JWTs are stateless with 24-hour expiry; roles include super_admin, admin, and user.
- File validation uses MIME allow-list, magic-byte checks, size caps, tenant quotas, and storage cleanup on delete.
- Channel security includes Twilio signature validation, Slack signature checks, Teams token validation, callback secrets, and MCP API-key auth.

D. Evaluation Detail

Run	Scope	Key result
Local full RAGAS	148 cases, local n8n eval webhook	faithfulness 0.782; relevancy 0.843; precision 0.940; recall 0.935; citation and abstention 1.000.
Backend DB-backed	5 Kubernetes cases through /chat/query	sources saved in chat_messages.sources; relevancy 0.940; precision 0.918; recall 1.000; faithfulness 0.638.
Public Railway smoke	5 direct public n8n cases	source return 1.000, average 3 sources/case, metadata coverage 1.000, no retries, mean latency 18.52 s.
Evaluator tooling	Batch-capable runner	Supports direct n8n URL, retries for 408/429/5xx/non-JSON, observability, JSON/CSV/MD/HTML reports.

The dataset spans Kubernetes troubleshooting, bug reporting, IT helpdesk, customer support, employee onboarding, developer documentation, incident response, compliance policy, education assistance, healthcare administration, legal-document navigation, equipment maintenance, and sales enablement. The strongest per-application faithfulness scores were legal navigation (1.000) and developer documentation (0.905); weaker domains identify where prompt and retrieval tuning should focus.

E. Module Work Completed

Module	Contribution
M1	Integration, architecture, branch workflow, locked stack, deployment coordination.
M2	Auth/core, tenants/users, JWT security, seed/admin flows.
M3	Document and chat APIs, storage, validation, n8n client integration, conversations.
M4	Unified message service, Slack adapter, channel routing, title derivation.
M5/M6	n8n ingestion, retrieval, reranking, generation, ephemeral workflows, purge.
M7	Postgres/pgvector schema, migrations, HNSW indexes, infrastructure notes.
M8/M9	Admin portal and chat portal UX, citations, status, user/document screens.
M10	RAGAS suite, synthetic corpora, reporting, public webhook batching.
M11-M14	Docs/demo, WhatsApp, onboarding, Microsoft Teams integration.

F. Limitations and Next Work

- Faithfulness did not yet meet the 0.85 RAGAS target, despite strong retrieval metrics. Add stricter post-generation validation and grounded retry loops before final user delivery.
- The public production endpoint needs a completed batched RAGAS run before claiming full production quality parity with the local evaluation.
- Ephemeral retrieval currently records faithfulness as null in its lean v1 path; add the Gemini self-check and fallback used by the main retrieval design.
- Future work includes hybrid keyword plus vector retrieval, structured chunking with better page metadata, external document synchronization, and voice interfaces.